UNIVERSITE CHOUAIB DOUKKALI
ECOLE NATIONALE DES SCIENCES APPLIQUEES D'EL JADIDA

جامعة شعيب الدكالي
Université Chouaib Doukkali

Cybersécurité & Confiance Numérique (CCN)

Algorithmique et programmation avancée avec Python

Pr. Fatima zahra SALMAM
salmam.f@ucd.ac.ma

python

1

Partie VIII :

- ✓Complexité algorithmique
- ✓Les algorithmes de tri

2

Motivation

- Un algorithme a pour objectif la résolution d'un problème.
- Est-ce que l'algorithme donne...
  1. une réponse ? **terminaison**
  2. la bonne réponse ? **correction**
  3. la bonne réponse en un temps acceptable ? **complexité**

3

Motivation

- On veut qu'un algorithme soit **correct**, mais aussi **efficace**, c'est-à-dire :
  - ✓**rapide** (en termes de temps d'exécution) ;
  - ✓**économe** en ressources (espace de stockage, mémoire utilisée) ;
- On a donc besoin d'outils qui nous permettront d'évaluer la qualité **théorique** des algorithmes proposés ;
- On se focalisera sur le temps d'exécution ;

4

Exemple : deux méthodes de recherche dans les listes

- Voici deux méthodes vérifiant si un élément appartient à une liste triée :

Version courte

```
def f(T, x):
    if x in T:
        return True
    return False
```

Version longue

```
def g(T, x, a=0, b=None):
    if b is None:
        b = len(T)-1

    if a > b:
        return False

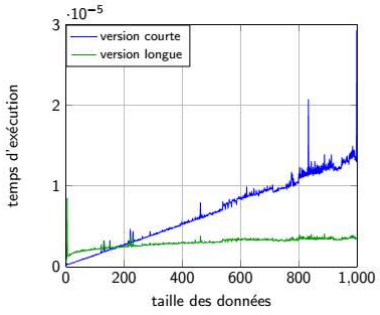
    p = (a + b) // 2
    if T[p] == x:
        return True
    if T[p] > x:
        return g(T, x, a, p-1)
    return g(T, x, p + 1, b)
```

Laquelle choisir ?

5

Exemple : deux méthodes de recherche dans les listes

- La mesure du temps d'exécution en pratique nous aide :



... mais ne nous dit pas pourquoi l'une est meilleure que l'autre.

6

Théorie de la complexité

- La **théorie de la complexité** (algorithmique) vise à répondre à ces besoins;
- Elle permet :
  1. de classer les problèmes selon leur **difficulté** ;
  2. de classer les algorithmes selon leur **efficacité**
  3. de **comparer** les algorithmes résolvant un problème donné afin de faire un choix éclairé sans devoir les implémenter ;

7

Evaluation de la rapidité d'un algorithme

- On ne mesure pas la durée en heures, minutes, secondes, ... :
  1. cela impliquerait d'implémenter les algorithmes qu'on veut comparer ;
  2. de plus, ces mesures ne seraient pas pertinentes car le même algorithme sera plus rapide sur une machine plus puissante ;
- Au lieu de ça, on utilise des **unités de temps abstraites** proportionnelles au nombre d'opérations effectuées ;
- Au besoin, on pourra ensuite adapter ces quantités en fonction de la machine sur laquelle l'algorithme s'exécute ;

8

Calcul du temps d'exécution

- Chaque **instruction basique** consomme une unite de temps ; (affectation d'une variable, comparaison, +, -, *, /, ...)
- Chaque **itération** d'une boucle rajoute le nombre d'unités de temps consommées dans le corps de cette boucle ;
- Chaque **appel de fonction** rajoute le nombre d'unités de temps consommées dans cette fonction ;
- Pour avoir le nombre d'opérations effectuées par l'algorithme, on additionne le tout ;

9

Exemple : calcul de la factorielle de n ϵ N

- Le programme suivant calcule n! = n*(n-1)*(n-2)*…*2*1 (avec 0! = 1) :

Exemple (factorielle de n)

```
def factorielle(n):
    fact = 1                          initialisation : 1
    i = 2                             initialisation : 1
    while i <= n:                     itérations : au plus n − 1
        fact = fact * i               multiplication + affectation : 2
        i = i + 1                     addition + affectation : 2
    return fact                       renvoi d'une valeur : 1
```

- On a aussi un test à chaque itération ;
- Nombre total d'opérations :

1+1+(n-1)*5+1=5n-2

10

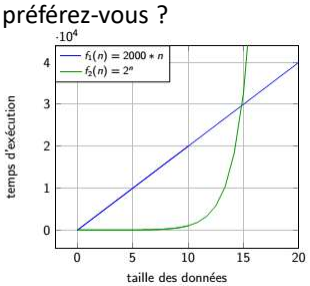
Complexité algorithmique

- La complexité d'un algorithme est une mesure de sa performance **asymptotique** dans le pire cas ;
- Que signifie **asymptotique** ?
  - → on s'intéresse à des données très grandes ;
  - Pourquoi ?
    - → les petites valeurs ne sont pas assez informatives ;
- Que signifie "dans le pire cas" ?
  - → on s'intéresse à la performance de l'algorithme dans les situations où le problème prend le plus de temps à résoudre ;
  - Pourquoi ?
    - → on veut être sûr que l'algorithme ne prendra jamais plus de temps que ce qu'on a estimé ;

11

Intuition : comportement asymptotique

- Soit :
  1. un problème à résoudre sur des données de taille n, et
  2. deux algorithmes résolvant ce problème en un temps f1(n) et f2(n), respectivement ;
- Quel algorithme préférez-vous ?



- La courbe verte semble correspondre à un algorithme beaucoup plus efficace ...
- ... mais seulement pour de très petites valeurs

12

La notation O (.) : motivation

- Les calculs à effectuer pour évaluer le temps d'exécution d'un algorithme peuvent parfois être longs et pénibles ;
- De plus, le degré de précision qu'ils requièrent est souvent inutile ;
- On aura donc recours à une **approximation** de ce temps de calcul, représentée par la notation O (.) ;

13

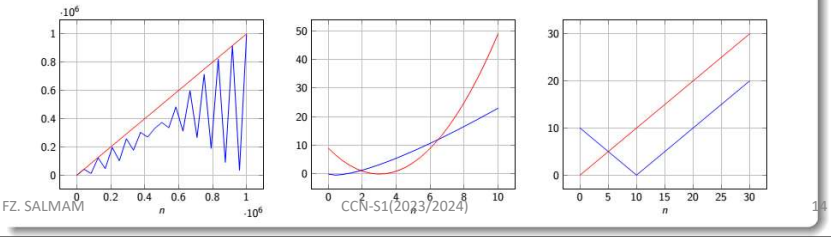
Notation

- Soit n la taille des données à traiter ; on dit qu'une fonction f(n) est **en** O(g (n)) ("**en grand** O de g(n)") si :

$$\exists n_0 \in N, \exists c \in R, \forall n \geq n_0 : |f(n)| \leq c|g(n)|$$

- Autrement dit : f(n) est en O(g(n)) s'il existe un seuil à partir duquel la fonction f(.) est toujours dominée par la fonction g(.), à une constante multiplicative fixée près ;

Exemple (quelques cas où $f(n) = O(g(n))$)



14

Exemples d'utilisation de O (.)

- On calcule le temps d'exécution comme avant, mais on effectue les simplifications suivantes :
  1. on oublie les constantes multiplicatives (elles valent 1) ;
  2. on annule les constantes additives ;
  3. on ne retient que les termes dominants ;

Exemple (simplifications)

Soit un algorithme effectuant $g(n) = 4n^3 - 5n^2 + 2n + 3$ opérations ;

1. on remplace les constantes multiplicatives par 1 : $1n^3 - 1n^2 + 1n + 3$
2. on annule les constantes additives : $n^3 - n^2 + n + 0$
3. on garde le terme de plus haut degré : $n^3 + 0$

et on a donc $g(n) = O(n^3)$.

15

Règles de calculs : combinaison des complexités

- Les instructions de base prennent un temps constant, noté O(1) ;
- On additionne les complexités d'opérations en séquence :

  O (f1(n)) + O (f2(n)) = O (f1(n) + f2(n))
- Pour les branchements conditionnels on prend le Max :

Exemple

```
if <condition>:          O(g(n))
    # instructions (1)   O(f1(n))
else:
    # instructions (2)   O(f2(n))
```

$=$ O (g(n) + O (max(f1(n), f2(n)))

16

Règles de calculs : combinaison des complexités

- Dans les boucles, on multiplie la complexité du corps de la boucle par le nombre d'itérations ;
- La complexité d'une boucle *while* se calcule comme suit :

Exemple

```
# en supposant qu'on a m itérations
while <condition>:        O(g(n))
    # instructions        O(f(n))
```

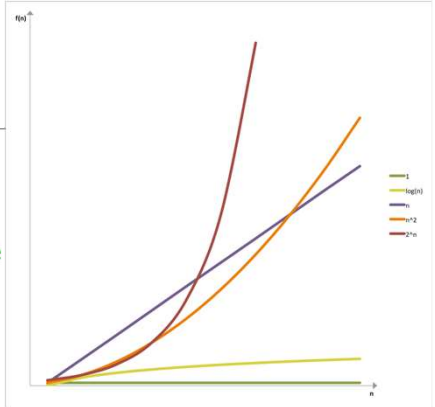
$$\left.\begin{matrix} O(g(n)) \\ O(f(n)) \end{matrix}\right\} = O(m * (g(n) + f(n)))$$

17

Quelques classes de complexité

- On peut donc "ranger" les fonctions équivalentes dans la même classe ;
- Voici quelques classes fréquentes de complexité (par ordre croissant en termes de O(.)) :

Complexité	Classe
$O(1)$	constant
$O(\log n)$	logarithmique
$O(n)$	linéaire
$O(n \log n)$	sous-quadratique
$O(n^2)$	quadratique
$O(n^3)$	cubique
$O(2^n)$	exponentiel



18

Quelques classes de complexité: log(n)

Définition du logarithme en base 2 :

- Le logarithme en base 2 d'un nombre n est le nombre d'itérations nécessaires pour diviser n par 2 jusqu'à atteindre 1 (ou un nombre proche de 1).
- Mathématiquement, $\log_2(n) = k$, où $2^k = n$.
- Cela signifie que 2 élevé à la puissance k équivaut à n.
- À chaque étape, si on suppose que la taille de la liste à examiner =n
- La division par 2 est effectué n fois.

```
def g(T, x, a=0, b=None):
    if b is None:
        b = len(T)-1

    if a > b:
        return False

    p = (a + b) // 2
    if T[p] == x:
        return True
    if T[p] > x:
        return g(T, x, a, p-1)
    return g(T, x, p + 1, b)
```


19

Quelques classes de complexité: nlog(n)

- La complexité **n log(n)** est couramment associée à des algorithmes rapides mais un peu plus coûteux que les algorithmes linéaires.
- Elle est souvent observée dans des algorithmes de tri efficaces tels que le tri rapide (Quicksort) et le tri fusion (Merge sort).
- Cette complexité combine une croissance linéaire **n** avec la croissance logarithmique **log(n)**, ce qui en fait des algorithmes efficaces pour des ensembles de données relativement importants.

20

Quelques classes de complexité: $O(2^n)$

- La complexité exponentielle, notée généralement **$O(2^n)$**, se réfère à une augmentation très rapide du temps d'exécution d'un algorithme en fonction de la taille des données d'entrée.
- Voici une fonction récursive qui calcule la puissance d'un nombre x à la puissance n.
- Dans cette fonction, pour calculer x^n, elle effectue une multiplication récursive n fois. Ainsi, pour n égal à 5 dans l'exemple, la fonction effectuera 5 multiplications successives (2 * 2 * 2 * 2 * 2) pour calculer 2^5.

```python
def puissance(x, n):
    if n == 0:
        return 1
    else:
        return x * puissance(x, n - 1)

# Exemple d'utilisation
resultat = puissance(2, 5)  # Calcul de 2^5
print("2^5 =", resultat)
```


Complexité et temps d'exécution

- Temps de calcul pour des données de taille 1 million

flops \ complexité	ln(n)	n	n²	2^n
10^6	0,013ms	1s	278 heures	10000 ans
10^9	0,013µs	1ms	¼ heure	10 ans
10^{12}	0,013ns	1µs	1s	1 semaine

Partie VIII :

✓Complexité algorithmique

✓Les algorithmes de tri

23

Le tri

- **Problème** : étant donné un tableau d'entiers T , trier T dans l'ordre croissant.
- Problème connu
- Grande richesse conceptuelle :
  - Des algorithmes basés sur des idées et des structures de données très différentes. . .
  - Des complexités différentes.
  - Des algorithmes optimaux.

24

Tri par sélection

- Le **tri par sélection** C'est sans doute la méthode de tri la plus élémentaire. On cherche le plus petit élément de la liste, puis on le place au début de la liste. Cette opération est répétée pour la liste restante, jusqu'à ce que tous les éléments soient dans l'ordre.

entrée : $T[1, n]$ est un tableau d'entiers, $n \geq 1$.
résultat : les éléments de T sont ordonnés par ordre croissant.
début
  pour $i := 1$ **à** $n - 1$ **faire**
    $j := \text{IndMin}(T[i, n])$ # indice du plus petit élément de $T[i, n]$
    permuter $T[i]$ et $T[j]$

IndMin(T [i , n]) : retourne l'indice du plus petit élément de {T [i], T[i+1], . . . , T[n]}.

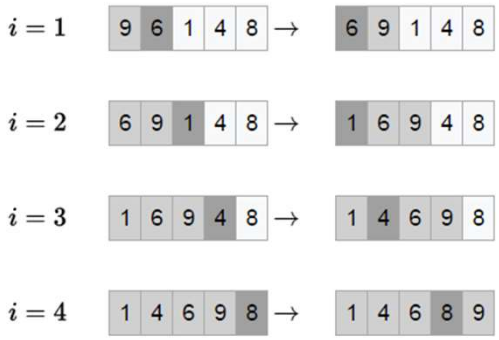
Tri par insertion

- C'est le tri du joueur de cartes. On considère que les éléments de la liste à trier sont donnés un par un. Avant de recevoir l'élément i on a déjà trié les éléments 1,..., i−1 . On insère alors l'élément i à la position appropriée.

entrée : $T[1, n]$ est un tableau d'entiers, $n \geq 1$.
résultat : les éléments de T sont ordonnés par ordre croissant.
début
  pour $i := 2$ **à** n **faire**
    $j := i$ # indice de l'élément à insérer dans $T[1, i - 1]$
    $v := T[i]$ # valeur de l'élément à insérer
    tant que $j > 1$ *et* $v < T[j - 1]$ **faire**
      $T[j] := T[j - 1]$ # $j - 1$ est une case libre
      $j := j - 1$ # j est une case libre
    $T[j] := v$

Tri par insertion

- **Exercice**: Exécuter le tri par insertion sur le tableau T=[9,6,1,4,8].



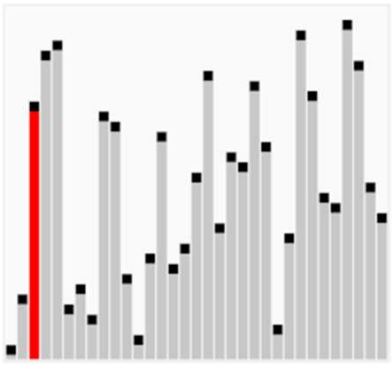
Tri par transposition (tri à bulle)

- Consiste à parcourir le tableau, tant qu'il n'est pas trié, et à permuter les couples d'éléments consécutifs mal ordonnés. On sait que le tableau est trié si lors d'un parcours, aucun couple d'éléments n'a été permuté.

entrée : $T[1, n]$ est un tableau d'entiers, $n \geq 1$.
résultat : les éléments de T sont ordonnés par ordre croissant.
début
  $M := n - 1$
  tant que *le tableau n'est peut-être pas trié* **faire**
    pour $i := 1$ **à** M **faire**
      si $T[i] > T[i + 1]$ **alors**
        permuter $T[i]$ et $T[i + 1]$
    $M = M - 1$

Tri par transposition (tri à bulle)

- Consiste à parcourir le tableau, tant qu'il n'est pas trié, et à permuter les couples d'éléments consécutifs mal ordonnés. On sait que le tableau est trié si lors d'un parcours, aucun couple d'éléments n'a été permuté.



29

Tri par transposition

- **Exercice**: Exécuter le tri à bulle sur le tableau T=[9,6,1,4,8].

30

Tri par fusion

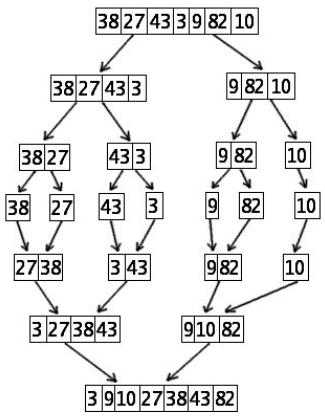
- L'idée du tri fusion est simple : on divise en deux moitiés la liste à trier, on trie chacune d'elles, puis on fusionne les deux moitiés pour reconstituer la liste complète triée.

entrée : $T[m, n]$ est un tableau d'entiers, $m \leq n$.
résultat : les éléments de T sont ordonnés par ordre croissant.
début
  si $m < n$ **alors**
    TriFusion($T[m, \lfloor \frac{m+n}{2} \rfloor]$)
    TriFusion($T[\lfloor \frac{m+n}{2} \rfloor + 1, n]$)
    Interclassement(T[m, n])

31

Tri par fusion

- **Exemple**: Exécuter le tri par fusion sur le tableau T=[38,27,43,3,9,82,10].



32

Tri rapide

- Un autre tri récursif. . . plus efficace en pratique.
- Etant donné un tableau de T [1, . . . , n]:
  - si n = 1, retourner le tableau T .
  - sinon :
    - Choisir un élément (le “pivot”) p dans T
    - Placer les éléments inférieurs à p au début de T
    - Placer p à sa place dans T
    - Placer les éléments supérieurs é p é la fin de T
    - Trier la première partie de T puis la seconde. . .

  (plus de fusion !)

33

Tri rapide

- Un autre tri récursif. . . plus efficace en pratique.
- Etant donné un tableau de T [1, . . . , n]:
  - si n = 1, retourner le tableau T .
  - sinon :
    - Choisir un élément (le “pivot”) p dans T
    - Placer les éléments inférieurs à p au début de T
    - Placer p à sa place dans T
    - Placer les éléments supérieurs à p à la fin de T
    - Trier la première partie de T puis la seconde. . .

  (plus de fusion !)

34

Tri rapide

- **Exemple**:

20, 15, 10, 35, 19, 13, 5, 3, 12, 7, 16, 40, 25, 38

15, 10, 19, 13, 5, 3, 12, 7, 16, 20, 35, 40, 25, 38

à trier ! à trier !

35